

Pixrup Pricing Structure (Locked & Enforced)

Tier	Price	Type	Features	Enforcement Logic
Micro	\$5 one-time	Pay-per-use	- Upload 1 file (screenshot, PDF, etc.) - AI reads & narrates text (TTS) - Download MP3/PDF - No account required	- No login → no access to dashboard - Single transaction via Stripe checkout - No API key created - Download link expires after 24h
Starter	\$99 / month	Subscription	- Access to dashboard + project history - Up to 50 documents per month - Basic support - Pixrup branding always visible	- <code>plan_tier = PRICE_STARTER</code> - Middleware blocks “Pro” or “Enterprise” features - DB column <code>used_docs</code> counted monthly - Stripe cancels access if unpaid
Pro	\$199 / month	Subscription	- Everything in Starter + - Unlimited documents - Logo upload + brand color - Priority support (chat/email) - AI “Before & After” photo rendering (Renovate AI) - Still shows “Powered by Pixrup” footer	- Feature flag: <code>branding_enabled = true</code> - Middleware: only Pro+ can hit <code>/branding</code> routes - Pixrup footer forced in PDF template - Limit 20 renders/month via DB counter
Enterprise	\$499 / month	Subscription	- All Pro features + - Full white-label PDFs (no footer) - Custom logo, colors, contact block - Admin panel for multiple users - API key access for automation - Dedicated account manager - Data retention & audit logs	- <code>white_label = true</code> - <code>show_powered_by = false</code> by default - Access to <code>/api/*</code> routes restricted via plan middleware - <code>max_users_per_org = 10</code> (configurable)

□ □ Technical Enforcement (Laravel)

1 □ Plan Tier Check Middleware

```
class CheckPlanTier {    public function handle($request, Closure $next, ...$tiers) {        $user = $request->user();    if (!$user || !in_array($user->plan_tier, $tiers)) {        return
```

```

redirect()->route('pricing')
}          return $next($request);    } }    ->with('error', 'Upgrade your plan to access this feature.');
```

2 Register Middleware

app/Http/Kernel.php

```
'plan' => \App\Http\Middleware\CheckPlanTier::class,
```

3 Protect Routes

```
// Only Pro & Enterprise Route::middleware(['auth','plan:PRICE_PRO,PRICE_ENTERPRISE']) ->group(function
() {
    Route::get('/branding', [BrandingController::class, 'show']);
    Route::post('/branding', [BrandingController::class, 'save']);
}); // Enterprise only
Route::middleware(['auth','plan:PRICE_ENTERPRISE']) ->group(function () {
    Route::get('/api-keys', [ApiController::class, 'index']);
});
```

4 Quota & Limit Enforcement

Add usage tracking:

```
Schema::table('users', function (Blueprint $t) {
    $t->integer('used_docs')->default(0);
    $t->date('usage_reset_at')->nullable();
});
```

Cron job daily:

```
php artisan schedule:run
```

In app/Console/Kernel.php:

```
$schedule->call(function() {
    User::where('usage_reset_at', '<', now())->update(['used_docs' => 0, 'usage_reset_at' => now()->addMonth()]);
});
```

When user generates a report:

```
if ($user->plan_tier === 'PRICE_STARTER' && $user->used_docs >= 50) {      return back()->with('error','Monthly limit reached. Upgrade to Pro.')} $user->increment('used_docs');
```

□ How to Prevent Tier Abuse

1. **Route Middleware:** only tiers allowed can access route features.
 2. **Blade Logic:** hide UI buttons if user's plan doesn't qualify.
 3. **PDF Templates:** always check plan before removing watermark.
 4. **Backend Validations:** double-check before action (not just UI-level).
 5. **API Keys:** generated only for Enterprise (/api-keys route locked).
 6. **Usage Counters:** Starter's doc counter prevents overuse.
-

□ Optional Enhancements

Feature	Who Gets It	Implementation
Audit Log (Enterprise)	Enterprise only	Table logs w/ each event + user_id
Multi-User Access	Enterprise	companies table + company_id foreign key
Custom Domain PDF Branding	Enterprise	Allow cdn.enterprise.pixrup.com subdomains
Webhook Integrations (Zapier, Make.com)	Pro+	API endpoints restricted via token
AI Voiceover Style Selector	Pro+	Dropdown stored as user pref (voice_style)

□ Summary

Tier	Price	Key Access
Micro (\$5)	One-time	Single AI report, expires 24h

Tier	Price	Key Access
Starter (\$99/mo)	Subscription	Dashboard + 50 docs/month
Pro (\$199/mo)	Subscription	Unlimited docs + Branding tab
Enterprise (\$499/mo)	Subscription	Full white-label + API access + team features

□ What You (Amar) Should Do

1. Confirm these 4 plans exist in **Stripe** (products + price IDs).
 2. Make sure `plan_tier` column syncs from Stripe → Laravel (via webhook).
 3. Tell Sagit:
 - Add `CheckPlanTier` middleware.
 - Use `used_docs` column for Starter limits.
 - Protect `/branding` and `/api/*` routes.
 - Hide Enterprise-only UI for lower tiers.
-

If you want, I can now:

- Create the **exact Laravel file structure + route map** for this pricing enforcement,
- Or write the **Blade pricing page** that shows locked/unlocked badges dynamically by plan.

Which one do you want next — the **route logic file structure** or the **front-end pricing page** version?

You said:

oh my god yes please

ChatGPT said:

□ Perfect, Amar — let's finish this right. Here's **Part 2** of your full developer + design blueprint: the **Laravel route logic file structure + dynamic front-end pricing page**.

Everything below is ready to drop into your project — it connects your Stripe tiers, enforces access control, and renders a gorgeous upgrade-locked pricing UI.

□ 1. Laravel File & Route Structure

□ routes/web.php

```
// Authentication routes already exist use App\Http\Controllers\{PricingController, BillingController,
BrandingController, ApiController}; // PUBLIC Route::get('/', fn() => view('home'))->name('home');
Route::get('/pricing', [PricingController::class, 'show'])->name('pricing'); // CHECKOUT / BILLING
Route::middleware('auth')->group(function () {      Route::post('/checkout/{tier}',
[BillingController::class, 'checkout'])->name('billing.checkout');      Route::get('/billing/success',
[BillingController::class, 'success'])->name('billing.success');      Route::get('/billing/cancel',
[BillingController::class, 'cancel'])->name('billing.cancel'); }); // STRIPE WEBHOOK
Route::post('/stripe/webhook', \Laravel\Cashier\Http\Controllers\WebhookController::class); // PRO +
ENTERPRISE Route::middleware(['auth', 'plan:PRICE_PRO,PRICE_ENTERPRISE'])->group(function () {
Route::get('/branding', [BrandingController::class, 'show'])->name('branding.show');
Route::post('/branding', [BrandingController::class, 'save'])->name('branding.save'); }); // ENTERPRISE
ONLY Route::middleware(['auth', 'plan:PRICE_ENTERPRISE'])->group(function () {      Route::get('/api-keys',
[ApiController::class, 'index'])->name('api.keys'); });
```

□ Middleware

app/Http/Middleware/CheckPlanTier.php

```
namespace App\Http\Middleware; use Closure; class CheckPlanTier {      public function handle($request,
Closure $next, ...$tiers) {          $user = $request->user();          if (!$user || !in_array($user-
>plan_tier, $tiers)) {              return redirect()->route('pricing')              ->with('error',
'Upgrade to access this feature.');
```

```
};          return $next($request);      } }
```

Register in Kernel.php:

'plan' => \App\Http\Middleware\CheckPlanTier::class,

□ 2. Front-End Pricing Page (Blade)

File: resources/views/billing/pricing.blade.php

This shows every plan, features, locked badges, and upgrade buttons using your PRICE_* vars.

```
@extends('layouts.app') @section('content') <div class="container mx-auto text-center py-10"> <h1
class="text-4xl font-bold mb-10 text-gray-800">Choose Your Pixrup Plan</h1> <div class="grid grid-cols-1
md:grid-cols-4 gap-6"> <!-- Micro --> <div class="border rounded-2xl shadow p-6"> <h2
class="text-xl font-bold mb-2">Micro</h2> <p class="text-3xl font-bold mb-4">$5<span class="text-base
text-gray-500"> one-time</span></p> <ul class="text-left text-sm mb-6"> <li>• Upload 1 file
(snapshot or PDF)</li> <li>• AI read & speech (MP3/PDF)</li> <li>• 24h download
expiry</li> </ul> <form method="POST" action="{{ route('billing.checkout','micro') }}">
@csrf <button class="btn-primary w-full bg-blue-600 text-white rounded-lg py-2">Buy Now</button>
</form> </div> <!-- Starter --> <div class="border rounded-2xl shadow p-6"> <h2
class="text-xl font-bold mb-2">Starter</h2> <p class="text-3xl font-bold mb-4">$99<span class="text-
base text-gray-500"> /mo</span></p> <ul class="text-left text-sm mb-6"> <li>• Dashboard +
history</li> <li>• 50 documents / month</li> <li>• Pixrup branding</li> <li>• Email
support</li> </ul> <form method="POST" action="{{ route('billing.checkout','starter') }}">
@csrf <button class="btn-primary w-full bg-blue-600 text-white rounded-lg py-2">Get
Starter</button> </form> </div> <!-- Pro --> <div class="border rounded-2xl shadow-lg p-
6 bg-blue-50"> <h2 class="text-xl font-bold mb-2">Pro</h2> <p class="text-3xl font-bold mb-
4">$199<span class="text-base text-gray-500"> /mo</span></p> <ul class="text-left text-sm mb-6">
<li>• Unlimited documents</li> <li>• Branding tab unlocked</li> <li>• 20 AI renders/mo</li>
<li>• Priority chat support</li> <li>• "Powered by Pixrup" footer</li> </ul> <form
method="POST" action="{{ route('billing.checkout','pro') }}"> @csrf <button class="btn-
primary w-full bg-blue-600 text-white rounded-lg py-2">Go Pro</button> </form> </div> <!--
Enterprise --> <div class="border rounded-2xl shadow-xl p-6 bg-gray-800 text-white"> <h2
class="text-xl font-bold mb-2">Enterprise</h2> <p class="text-3xl font-bold mb-4">$499<span
class="text-base text-gray-400"> /mo</span></p> <ul class="text-left text-sm mb-6"> <li>•
Full white-label PDFs</li> <li>• Custom logo, color & contact</li> <li>• API access &
multi-user admin</li> <li>• Dedicated manager</li> <li>• Audit logs & retention</li>
</ul> <form method="POST" action="{{ route('billing.checkout','enterprise') }}"> @csrf
```

```
<button class="btn-primary w-full bg-blue-500 text-white rounded-lg py-2">Upgrade to Enterprise</button>
</form>      </div>    </div> </div> @endsection
```

□ 3. How to Block Feature Bypass

- **Routes:** Guarded with the `CheckPlanTier` middleware (users can't call hidden endpoints).
- **Blades:** Hide buttons with:

```
@if(auth()->user()->plan_tier === 'PRICE_ENTERPRISE')    <span class="badge bg-green-600">Your Plan</span>
@else    <form method="POST" ...>Upgrade</form> @endif
```

- **Server-side validation** (backend double-checks every action).
 - **Stripe Webhook Sync** keeps `plan_tier` current — if a payment fails, plan auto-downgrades.
 - **Usage limits** (Starter) block document creation beyond quota.
-

□ 4. Dev Notes for Sagit

- Use your existing layout system (Tailwind + Blade components).
 - Wrap `/branding` + `/api-keys` with middleware.
 - Add `used_docs` column for Starter limits.
 - Reuse DomPDF logic already in place for the PDFs.
 - Add `plan_tier` checks before showing “Export Report” or “Download AI Render.”
-

□ 5. Business Control

Plan	Monthly You Earn (After Stripe) Margin %		
Micro	\$5	~\$4.70	94%
Starter	\$99	~\$95.13	96%
Pro	\$199	~\$191.23	96%
Enterprise	\$499	~\$479.53	96%

1. Middleware Enforcement

□ **File:** `app/Http/Middleware/CheckFeatureLimit.php`

This middleware makes sure no feature (like AI rendering or PDF generation) is used beyond allowed quota or plan.

```
namespace App\Http\Middleware; use Closure; use Illuminate\Support\Facades\Auth; class CheckFeatureLimit
{
    public function handle($request, Closure $next, $feature) {
        $user = Auth::user();
        if (!$user) {
            return redirect()->route('login')->with('error', 'Login required.');
```


Register it:

```
app/Http/Kernel.php
```

```
'feature' => \App\Http\Middleware\CheckFeatureLimit::class,
```

□ □ 2. Model Trait for Usage Tracking

□ **File:** `app/Models/Traits/TracksUsage.php`

This handles monthly resets and easy usage increments for any feature.

```
namespace App\Models\Traits; use Carbon\Carbon; trait TracksUsage {      public function
incrementUsage($field)      {          $this->checkUsageReset();          $this->increment($field);      }
public function checkUsageReset()      {          if (!$this->usage_reset_at || Carbon::now()-
>greaterThan($this->usage_reset_at)) {          $this->update([          'used_docs' => 0,
'used_renders' => 0,          'usage_reset_at' => Carbon::now()->addMonth()          ]);
}      } }
```

□ 3. Add Fields in Users Table

Migration snippet:

```
Schema::table('users', function (Blueprint $table) {      $table->integer('used_docs')->default(0);
$table->integer('used_renders')->default(0);      $table->date('usage_reset_at')->nullable(); });
```

□ 4. Usage in Controller

Whenever a user generates something:

```
use App\Models\Traits\TracksUsage; class ReportController extends Controller { public function
generate(Request $r) { $user = $r->user(); // enforce limit via middleware before
running this if ($user->plan_tier === 'PRICE_STARTER' && $user->used_docs >= 50) {
return back()->with('error', 'Limit reached. Upgrade to Pro.');
```

□ 5. Apply Middleware to Routes

```
// AI Render Route::post('/ai/render', [RenderController::class, 'process']) ->middleware(['auth',
'feature:ai_render']); // Generate PDF Route::post('/reports/generate', [ReportController::class,
'generate']) ->middleware(['auth', 'feature:generate_report']); // White-label Branding
Route::get('/branding', [BrandingController::class, 'show']) ->middleware(['auth',
'feature:white_label']);
```

□ 6. Scheduler for Automatic Monthly Reset

This ensures Starter and Pro users reset their counters monthly.

File: app/Console/Kernel.php

```
protected function schedule(Schedule $schedule) { $schedule->call(function () {
\App\Models\User::whereDate('usage_reset_at', '<=', now())->update(['used_docs' => 0,
'used_renders' => 0, 'usage_reset_at' => now()->addMonth(), ]); }->daily(); }
```

□ 7. Backend Fail-Safes

Protection	How It Works
Middleware Layer	Blocks user requests at route level (can't even reach controller).
Controller Double-Check	Increments usage only if limit not exceeded.
Database Counters	Tracks monthly usage per user.
Auto Reset	Cron job resets monthly limits.
Stripe Sync	Downgrades automatically if payment fails.
API Key Restriction	/api/* routes locked to PRICE_ENTERPRISE via middleware.

□ 8. Bonus — Helper Functions

Add to `app/helpers.php`:

```
function planName($plan) { return match($plan) {
    'PRICE_STARTER' => 'Starter ($99/mo)', 'PRICE_PRO' => 'Pro ($199/mo)', 'PRICE_ENTERPRISE'
=> 'Enterprise ($499/mo)', default => 'Unknown Plan' }; }
function hasFeature($user, $feature) {
    return match($feature) {
        'branding' => in_array($user->plan_tier,
        ['PRICE_PRO', 'PRICE_ENTERPRISE']), 'api' => $user->plan_tier === 'PRICE_ENTERPRISE',
        'render' => in_array($user->plan_tier, ['PRICE_PRO', 'PRICE_ENTERPRISE']), default => false }; }
```

□ Business Enforcement Recap

Feature	Starter	Pro	Enterprise
Docs / month	50	Unlimited	Unlimited
AI Renders	□	20/mo	Unlimited
Branding	□	Header + color	Full white-label
API Access	□	□	□
Team Admin	□	□	□
Support	Email	Priority	Dedicated

Feature	Starter	Pro	Enterprise
Reset	Monthly	Monthly	Monthly

PIXRUP PLAN & FEATURE TEST SUITE (For Developer QA)

☐ Purpose:

To verify every paid tier (Micro / Starter / Pro / Enterprise) behaves correctly — that users cannot bypass limits, generate beyond quotas, or access restricted routes.

☐ 1. SETUP PREPARATION

Before testing, Sagit must:

- ☐ Create **4 Stripe test products** with IDs:
PRICE_MICRO, PRICE_STARTER, PRICE_PRO, PRICE_ENTERPRISE
- ☐ Enable **Stripe test mode**.
- ☐ Ensure the database has these user columns:

`plan_tier, used_docs, used_renders, usage_reset_at`

- ☐ Run `php artisan schedule:run` once to verify the monthly reset logic.

□ 2. PLAN LOGIC TEST CASES

Case	Action	Expected Result
1□□ Micro Checkout	Visit <code>/pricing</code> → Click “Buy Now” on Micro Complete payment (Stripe test card) Try to access <code>/branding</code>	Redirect to Stripe checkout, no account required. User gets “Download MP3” link (expires after 24h). Redirect to pricing page with error: “Upgrade required.”
2□□ Starter Plan Registration	Register new user → <code>/billing/checkout/starter</code> Try creating 50 docs Wait for monthly cron reset Access <code>/branding</code>	Stripe checkout → success redirect <code>/dashboard</code> . Works until #50; 51st → error: “Monthly limit reached.” Counter resets (<code>used_docs = 0</code>). Redirect with “Upgrade to Pro.”
3□□ Pro Plan Upgrade	Upgrade Starter user via <code>/billing/checkout/pro</code> Access <code>/branding</code> Upload logo → generate PDF Generate 20 AI renders Access <code>/api-keys</code>	Plan tier changes → “Upgrade Successful.” Opens Branding tab successfully. PDF shows uploaded logo but still “Powered by Pixrup.” Works until #20; 21st → error “Render limit reached.” Redirect with “Enterprise only.”
4□□ Enterprise Plan Upgrade	Upgrade Pro user to <code>/billing/checkout/enterprise</code> Access <code>/api-keys</code> Generate PDF Generate unlimited renders Add team users (if multi-user added)	Plan tier updates instantly. Opens page (visible keys + create new). Full white-label (no footer). Works indefinitely (no limit). All members inherit Enterprise plan access.

□ 3. MIDDLEWARE TESTS

Middleware	Route	Expected Behavior
CheckPlanTier	<code>/branding</code> <code>/api-keys</code>	Accessible to Pro/Enterprise only. Enterprise only.

Middleware	Route	Expected Behavior
CheckFeatureLimit:generate_report	/reports/generate	Blocks Starter after 50 docs.
CheckFeatureLimit:ai_render	/ai/render	Blocks Starter entirely; Pro after 20 renders.
CheckFeatureLimit:white_label	/branding	Blocks everyone except Enterprise.

□ □ 4. STRIPE SYNCHRONIZATION TESTS

Scenario	Expected Behavior
New user completes checkout	Stripe webhook updates <code>plan_tier</code> .
Payment fails / cancelled	Stripe webhook downgrades plan to “Starter” (or “Free”).
Manual Stripe upgrade	<code>plan_tier</code> auto-updates on next webhook event.

Webhook command (for local dev):

```
stripe listen --forward-to localhost:8000/stripe/webhook
```

Webhook Handler (Laravel Cashier)

Located automatically at `/stripe/webhook` — ensures subscription sync.

□ 5. DATABASE STATE CHECKS

Run Tinker commands to confirm:

```
php artisan tinker >>> App\Models\User::first()->toArray()
```

□ **Expected Fields:**

```
{  "plan_tier": "PRICE_PRO",    "used_docs": 18,    "used_renders": 12,    "usage_reset_at": "2025-11-24T00:00:00Z" }
```

If testing Enterprise:

```
{  "plan_tier": "PRICE_ENTERPRISE",    "white_label": true }
```

□ 6. FRONT-END VISUAL CONFIRMATION

Plan	Pricing Page Label	Feature Availability
Micro	“Buy Now”	Only checkout link (no login)
Starter	“Get Starter”	Dashboard active, Branding tab hidden
Pro	“Go Pro”	Branding tab visible, 20 renders
Enterprise	“Upgrade to Enterprise”	All features unlocked, white-label active

□ 7. ATTEMPTED BYPASS TESTS

Exploit Attempt	Expected Response
Change plan_tier via DevTools	Ignored — backend middleware blocks routes
Direct URL /api-keys as Pro	Redirects with “Enterprise only.”
Manually call /branding as Starter	Redirect with “Upgrade required.”
F12 Edit “Powered by Pixrup” footer	Still injected at server level (PDFKit template)
Exceed render limit (Pro)	Error toast, log entry created.
Duplicate API key for Pro	403 Forbidden.

□ 8. LOGGING & MONITORING

Enable **LogRocket (Free Tier)** for:

- ☐ User plan tier logs
- ☐ API key creation
- ☐ Feature access attempts

Log each blocked action like:

```
Log::warning("Blocked feature access", [      'user' => $user->id,      'plan' => $user->plan_tier,      'feature' => $feature ]);
```

☐ 9. SUCCESS CRITERIA

- ☐ All features restricted by tier.
 - ☐ Stripe plan auto-sync verified.
 - ☐ Monthly reset functioning.
 - ☐ Branding + white-label visually correct.
 - ☐ Bypass attempts rejected at middleware level.
 - ☐ Admin can manually view user stats in DB.
 - ☐ No runtime errors in Laravel logs.
-

☐ 10. OPTIONAL AUTOMATION

Use **Laravel Dusk** or **PestPHP** for automated browser tests:

```
php artisan dusk
```

Sample Dusk test:


```
public function test_starter_limit_enforced() {      $user = User::factory()->create(['plan_tier' =>
'PRICE_STARTER', 'used_docs' => 50]);      $this->actingAs($user)      ->post('/reports/generate')
->assertRedirect('/pricing')      ->assertSessionHas('error'); }
```

□ FINAL VERIFICATION ORDER FOR SAGIT

1. Create test users → assign plan tiers.
2. Test each tier's access, limits, and UI.
3. Verify Stripe webhooks update plan correctly.
4. Check all redirects, limits, and logs.
5. Generate 1 Micro, 2 Starter, 2 Pro, and 1 Enterprise transaction → full flow test.
6. Record screen capture for Amar (proof of function).

1. PURPOSE

Your **Admin Dashboard** lets you:

- ☐ View all users & current plan tiers
- ☐ See who hit usage limits
- ☐ Manually reset usage or downgrade/upgrade plans
- ☐ Monitor Stripe subscription status
- ☐ Generate reports (active users, revenue, API usage)

☐ ☐ 2. RECOMMENDED STACK

Feature	Framework	Reason
Admin Panel	Filament (modern, free)	Faster UI, simple setup, Tailwind native
Analytics	Filament Widgets + Laravel Charts (Chart.js)	Lightweight, no extra cost
Auth	Same user table (admin role flag)	No second login system
Access Control	Middleware <code>is_admin</code>	Prevent non-admin users

☐ 3. SETUP STRUCTURE

☐ Add Admin Role to Users

Migration:

```
Schema::table('users', function (Blueprint $table) {  
    $table->boolean('is_admin')->default(false);  
});
```

□ Middleware: IsAdmin

app/Http/Middleware/IsAdmin.php

```
namespace App\Http\Middleware; use Closure; use Illuminate\Support\Facades\Auth; class IsAdmin {
public function handle($request, Closure $next) { if (!Auth::check() || !Auth::user()->is_admin) {
abort(403, 'Unauthorized.');
```

Register in Kernel.php:

```
'admin' => \App\Http\Middleware\IsAdmin::class,
```

□ Routes

```
Route::middleware(['auth','admin'])->prefix('admin')->group(function() { Route::get('/',
[AdminController::class, 'dashboard'])->name('admin.dashboard');
```

□ 4. FILAMENT DASHBOARD INSTALLATION

1□□ Install Filament

```
composer require filament/filament:"^3.0" php artisan make:filament-user
```

2□□ Protect Filament Access

Add to app/Providers/Filament/AdminPanelProvider.php:

```
use Filament\Facades\Filament; public function boot(): void { Filament::serving(function () {
if (auth()->check() && !auth()->user()->is_admin) { abort(403); } }); }
```

□ 5. DASHBOARD DESIGN

□ Users Table

Columns:

- ID
- Name
- Email
- Plan Tier
- Used Docs
- Used Renders
- Usage Reset Date
- Status (Active / Suspended)

Use Filament's `Tables` resource:

```
php artisan make:filament-resource User
```

Modify resource:

```
public static function table(Table $table): Table {    return $table        ->columns([
    TextColumn::make('id')->sortable(),                TextColumn::make('name')->searchable(),
    BadgeColumn::make('plan_tier')->colors([            'success' => 'PRICE_ENTERPRISE',
    'warning' => 'PRICE_PRO',                            'info' => 'PRICE_STARTER',                'gray' =>
    'PRICE_MICRO',                                     TextColumn::make('used_docs'),
    TextColumn::make('used_renders'),                    TextColumn::make('usage_reset_at')->date(),                ])
    ->filters([
        SelectFilter::make('plan_tier')                ->options([
        'PRICE_MICRO' => 'Micro',                        'PRICE_STARTER' => 'Starter',                'PRICE_PRO'
        => 'Pro',                                        'PRICE_ENTERPRISE' => 'Enterprise',                ]),                ])
    ->actions([
        Tables\Actions\EditAction::make(),
        Tables\Actions\Action::make('reset_usage')        ->label('Reset Usage')                -
        ->action(fn(User $u) => $u->update([                'used_docs' => 0,
```

```
'used_renderers' => 0,
]); }

'usage_reset_at' => now()->addMonth(),
    ])),
```

□ 6. ANALYTICS WIDGETS

Example Filament Widgets:

A. Active Users

```
class ActiveUsers extends StatsOverviewWidget {
    protected function getCards(): array {
        return [
            Card::make('Active Users', User::count()),
            Card::make('Enterprise Users',
                User::where('plan_tier', 'PRICE_ENTERPRISE')->count()),
            Card::make('Monthly Revenue', '$' .
                (User::sum('revenue_estimate'))),
        ];
    }
}
```

B. Usage Chart

```
class UsageChart extends ChartWidget {
    protected function getData(): array {
        $data =
        User::selectRaw('plan_tier, sum(used_docs) as total_docs')
            ->groupBy('plan_tier')
            ->pluck('total_docs', 'plan_tier');
        return [
            'datasets' => [
                [
                    'label' => 'Documents Generated',
                    'data' => $data->values(),
                    'backgroundColor' => ['#3b82f6', '#f59e0b', '#10b981', '#6366f1'],
                ],
            ],
            'labels' => $data->keys(),
        ];
    }
    protected function getType(): string {
        return 'bar';
    }
}
```

□ 7. STRIPE OVERVIEW PANEL

Show revenue and plan distribution via Filament's InfoWidget:

```
$revenue = \App\Models\User::all()->sum(fn($u) => match($u->plan_tier) {
    'PRICE_PRO' => 199,
    'PRICE_ENTERPRISE' => 499,
    default => 0 });
$revenue = \App\Models\User::all()->sum(fn($u) => match($u->plan_tier) {
    'PRICE_STARTER' => 99,
```

Display `$revenue` as “Estimated Monthly MRR”.

☐ 8. ADMIN CONTROLS

Task	Admin Action
Reset user usage	Button → <code>reset_usage</code> action
Downgrade plan	Edit → change <code>plan_tier</code>
Suspend account	<code>is_active = false</code>
View Stripe link	<code>stripe_customer_id</code> column
Export report	Filament export plugin (<code>filament/exports</code>)

☐ 9. SECURITY RULES

- ☐ Only `is_admin` users access `/admin` or Filament.
 - ☐ Filament session timeout 30 mins (in `config/session.php`).
 - ☐ All CRUD actions logged via Laravel `activity()` log.
-

☐ 10. RESULT

With this dashboard, you'll:

- ☐ See total MRR and plan mix instantly
- ☐ Reset user quotas in one click
- ☐ Spot heavy users before upgrades

- ❑ Track compliance of your limits system visually
- ❑ Operate Pixrup like a SaaS CEO ❑

Sagit already working in **Laravel**, which gives him:

- ❑ Routing system ❑
- ❑ Middleware system ❑
- ❑ Database migrations ❑
- ❑ Blade templating ❑

And the **Filament admin package** I mentioned is **Laravel-native** — it installs with one command, and comes with full documentation and prebuilt UI widgets (no design needed).

So Sagit doesn't have to "design" dashboards manually; Filament generates them instantly.

❑❑ WHAT SAGIT WILL NEED TO DO

Step	Task	Difficulty	Notes
1❑❑	Run <code>composer require filament/filament</code>	❑ Easy	Installs admin system automatically
2❑❑	Create an admin user (<code>is_admin=true</code>)	❑ Easy	Use Laravel Tinker or DB seed
3❑❑	Add middleware <code>IsAdmin</code>	❑ Easy	Copy-paste code I gave you
4❑❑	Run <code>php artisan make:filament-resource User</code>	❑ Easy	Creates full user management table
5❑❑	Add columns (plan, usage, etc.)	❑ Moderate	Copy from the code snippet above
6❑❑	Add Filament widgets for analytics	❑ Moderate	Chart.js + Filament built-in widgets
7❑❑	Secure routes (<code>/admin</code>)	❑ Easy	Middleware already set
8❑❑	(Optional) Stripe integration for live revenue	❑ Medium	Add Stripe API key + fetch totals

- ☐ = easy copy/paste
- ☐ = simple logic edits
- ☐ = advanced but straightforward if following docs

☐☐ WHAT YOU'LL GET AFTER HE BUILDS IT

When Sagit is done, you'll log into:

☐ **pixrup.com/admin**

and see:

- ☐ Dashboard cards: *Active Users*, *Enterprise Users*, *Monthly Revenue*
- ☐ Table of all users (plan tiers, usage counts, reset dates)
- ☐ Action buttons ("Reset Usage", "Downgrade", "Suspend")
- ☐ Charts showing total docs generated by plan
- ☐ Real-time revenue chart (if Stripe connected)

☐ SECURITY CHECKLIST FOR SAGIT

Step	Description	Why
☐ Middleware <code>is_admin</code>	Only you (Amar) can access admin	Prevent user sneaking into dashboard
☐ Hide Filament route	<code>/admin</code> path protected	Stops public scans
☐ Use your Google 2FA login	Enable for admin user	Protects from brute-force attacks
☐ Disable registration on Filament	Done in config	Keeps admin invite-only
☐ Laravel activity log	Log every admin action	Audit trail for transparency

☐ COST & SPEED

- ☐ **Filament:** Free, zero subscription.
- ☐ **Setup time:** ~1–2 days max (if Sagit copies everything above).
- ☐ **Ongoing cost:** \$0.
- ☐ **Hosting:** Works on your existing Hostinger VPS setup.

☐ BONUS OPTION

If you want a slightly more *corporate-looking* admin (like a SaaS dashboard with widgets & charts), Sagit can optionally install:

```
composer require bezhansalleh/filament-shield composer require pxlrbt/filament-excel composer require  
awcodes/filament-table-repeater
```

These add:

- ☐ Role-based permissions
- ☐ Export to Excel/CSV
- ☐ Chart dashboards

All are free & open-source, built for Filament.

PIXrup Admin Dashboard – Full Developer Setup Guide (Filament 3)

Goal: Build the admin system that lets you (Amar) monitor all users, plan tiers, usage quotas, and Stripe revenue — all in one secure dashboard.

☐ ☐ Step 1 – Install Filament

In terminal (inside project root):

```
composer require filament/filament:"^3.0" php artisan filament:install
```

This creates the Filament base panel (/admin route automatically added).

☐ Step 2 – Create an Admin User

In terminal:

```
php artisan tinker >>> $u = App\Models\User::find(1); >>> $u->is_admin = true; >>> $u->save();
```

☐ Now user ID 1 (you, Amar) can log in to /admin.

☐ Step 3 – Secure Admin Access

Create middleware:

app/Http/Middleware/IsAdmin.php

```
namespace App\Http\Middleware; use Closure; use Illuminate\Support\Facades\Auth; class IsAdmin {  
public function handle($request, Closure $next) { if (!Auth::check() || !Auth::user()->is_admin) {  
abort(403, 'Unauthorized.');
```

Register it in app/Http/Kernel.php:

```
'admin' => \App\Http\Middleware\IsAdmin::class,
```

□ Step 4 – Protect Filament Routes

Edit app/Providers/Filament/AdminPanelProvider.php:

```
use Filament\Facades\Filament; public function boot(): void { Filament::serving(function () {  
if (auth()->check() && !auth()->user()->is_admin) { abort(403); } }); }
```

This ensures only admins see the Filament UI.

□ Step 5 – Add Admin Routes

routes/web.php

```
Route::middleware(['auth','admin'])->prefix('admin')->group(function() { Route::get('/', fn() =>  
redirect('/admin'))->name('admin.dashboard'); });
```

□ Step 6 – Create Filament User Resource

Run:

```
php artisan make:filament-resource User
```

Then open:

```
app/Filament/Resources/UserResource.php
```

Replace table() method:

```
public static function table(Table $table): Table {    return $table        ->columns([
    TextColumn::make('id')->sortable(),                TextColumn::make('name')->searchable(),
    TextColumn::make('email'),                        BadgeColumn::make('plan_tier')                ->label('Plan')
->colors(['gray' => 'PRICE_MICRO',                    'info' => 'PRICE_STARTER',
'warning' => 'PRICE_PRO',                            'success' => 'PRICE_ENTERPRISE',                ]),
    TextColumn::make('used_docs')->label('Docs Used'),    TextColumn::make('used_renders')-
>label('Renders Used'),    TextColumn::make('usage_reset_at')->date()->label('Resets On'),
    ])        ->actions([                Tables\Actions\EditAction::make(),
    Tables\Actions\Action::make('Reset Usage')                ->icon('heroicon-o-refresh')        -
>action(fn($record) => $record->update([                'used_docs' => 0,
'used_renders' => 0,                'usage_reset_at' => now()->addMonth(),                ]))
->requiresConfirmation(),                ])        ->filters([                SelectFilter::make('plan_tier')-
>options([                'PRICE_MICRO' => 'Micro',                'PRICE_STARTER' => 'Starter',
'PRICE_PRO' => 'Pro',                'PRICE_ENTERPRISE' => 'Enterprise',                ]),                ]); }
```

☐ Now the admin can see all users, filter by plan, and reset usage.

☐ Step 7 – Add Dashboard Widgets

Run:

```
php artisan make:filament-widget ActiveStats php artisan make:filament-widget UsageChart
```

Widget 1 – ActiveStats

app/Filament/Widgets/ActiveStats.php

```
namespace App\Filament\Widgets; use App\Models\User; use Filament\Widgets\StatsOverviewWidget as
BaseWidget; use Filament\Widgets\StatsOverviewWidget\Card; class ActiveStats extends BaseWidget {
protected function getCards(): array { return [ Card::make('Active Users',
User::count()), Card::make('Enterprise Users', User::where('plan_tier','PRICE_ENTERPRISE')-
>count()), Card::make('Monthly Revenue', '$'.( User::sum(fn($u) => match($u-
>plan_tier) { 'PRICE_STARTER' => 99, 'PRICE_PRO' => 199,
'PRICE_ENTERPRISE' => 499, default => 0 }) ))), ];
} }
```

Widget 2 – UsageChart

app/Filament/Widgets/UsageChart.php

```
namespace App\Filament\Widgets; use App\Models\User; use Filament\Widgets\ChartWidget; class UsageChart
extends ChartWidget { protected function getData(): array { $data =
User::selectRaw('plan_tier, sum(used_docs) as total')
->groupBy('plan_tier')
->pluck('total','plan_tier'); return [ 'datasets' => [[ 'label' =>
'Docs Generated', 'data' => $data->values(), 'backgroundColor' =>
['#3b82f6','#f59e0b','#10b981','#6366f1'], ], 'labels' => $data->keys(),
]; } protected function getType(): string { return 'bar'; } }
```

□ These widgets will auto-appear on /admin home.

□ Step 8 – (Optional) Live Stripe Revenue Chart

Add your Stripe API key to .env:

```
STRIPE_SECRET=sk_test_*****
```

Install Stripe PHP SDK:

```
composer require stripe/stripe-php
```

Then create widget:

```
app/Filament/Widgets/StripeRevenue.php
```

```
use Stripe\Stripe; use Stripe\Subscription; use Filament\Widgets\StatsOverviewWidget\Card; class
StripeRevenue extends BaseWidget {    protected function getCards(): array    {
Stripe::setApiKey(env('STRIPE_SECRET'));    $subs = Subscription::all(['limit' => 50]);
$revenue = collect($subs->data)->sum('plan.amount') / 100;    return [    Card::make('Live
Stripe MRR', '$'.$revenue),    ];    } }
```

□ Step 9 – Activate Widgets

Open app/Filament/Pages/Dashboard.php and register:

```
protected function getWidgets(): array {    return [    ActiveStats::class,
UsageChart::class,    StripeRevenue::class,    ]; }
```

□ Step 10 – Security Hardening

- □ Only allow admins: `is_admin = true`
- □ Add 2FA for your admin account (Google Authenticator)
- □ Disable public Filament registration
- □ Add IP restriction (optional):

```
if (!in_array($request->ip(), ['your-static-ip'])) abort(403);
```

□ Step 11 – Final Verification

1 ☐ ☐ Log in as Amar → /admin

2 ☐ ☐ You should see:

- ☐ 3 widgets on top (Active Users, Enterprise Users, Monthly Revenue)
- ☐ Table of users with filters and “Reset Usage”
- ☐ Bar chart of total docs generated
- ☐ Optional Stripe Live MRR widget

3 ☐ ☐ Test by upgrading a user’s `plan_tier` → chart & totals auto-update.

STRIPE CUSTOMER HISTORY DASHBOARD

☐ 1. What It Does

Inside /admin, each user row gets a “**Stripe History**” tab showing:

- ☐ Current subscription plan
- ☐ Next renewal date
- ☐ Last payment status
- ☐ Stripe Customer ID (clickable link)
- ☐ Payment method type (card/bank)
- ☐ Total lifetime spend

All data is fetched live from your **Stripe API**.

☐ ☐ 2. Install Stripe SDK

If not already installed:

```
composer require stripe/stripe-php
```

Add to .env:

```
STRIPE_SECRET=sk_test_***** STRIPE_KEY=pk_test_*****
```

□ 3. Create a Stripe Service Helper

□ app/Services/StripeService.php

```
namespace App\Services; use Stripe\Stripe; use Stripe\Customer; use Stripe\Subscription; use
Stripe\Invoice; class StripeService { public function __construct() {
Stripe::setApiKey(env('STRIPE_SECRET')); } public function getCustomerData($customerId) {
try { $customer = Customer::retrieve($customerId); $subscriptions =
Subscription::all(['customer' => $customerId, 'limit' => 1]); $invoices =
Invoice::all(['customer' => $customerId, 'limit' => 5]); $sub = $subscriptions->data[0] ??
null; return [ 'customer' => $customer, 'subscription' => $sub,
'invoices' => $invoices->data, ]; } catch (\Exception $e) { return ['error'
=> $e->getMessage()]; } }
```

□ 4. Add Column in User Table

In your DB (or migration):

```
Schema::table('users', function (Blueprint $t) { $t->string('stripe_customer_id')->nullable(); });
```

When new users subscribe via Stripe Checkout (Laravel Cashier), this field is automatically filled.

□ 5. Create Filament Page – “Stripe History”

Run:

```
php artisan make:filament-page StripeHistory
```

Then edit:

```
□ app/Filament/Pages/StripeHistory.php
```

```
namespace App\Filament\Pages; use App\Models\User; use App\Services\StripeService; use
Filament\Pages\Page; class StripeHistory extends Page {     protected static ?string $navigationIcon =
'heroicon-o-credit-card';     protected static string $view = 'filament.pages.stripe-history';
protected static ?string $title = 'Customer Billing History';     public $user;     public $stripeData;
public function mount($record): void {         $this->user = User::find($record);         $service =
new StripeService();         $this->stripeData = $service->getCustomerData($this->user-
>stripe_customer_id);     } }
```

□ 6. Create the Blade View

```
□ resources/views/filament/pages/stripe-history.blade.php
```

```
<x-filament::page>     <div class="space-y-6">         <div class="bg-white rounded-xl p-6 shadow">             <h2
class="text-xl font-bold mb-4">Stripe Details for {{ $user->name }}</h2>
@if(isset($stripeData['error']))             <p class="text-red-600">{{ $stripeData['error'] }}</p>             @else
<div class="grid grid-cols-2 gap-4">                 <p><strong>Customer ID:</strong>                 <a
href="https://dashboard.stripe.com/test/customers/{{ $user->stripe_customer_id }}" target="_blank"
class="text-blue-600 underline">                 {{ $user->stripe_customer_id }}                 </a>
</p>                 <p><strong>Current Plan:</strong> {{ $user->plan_tier }}</p>                 <p><strong>Next
Renewal:</strong> {{ $stripeData['subscription']?->current_period_end ? date('Y-m-d',
$stripeData['subscription']->current_period_end) : 'N/A' }}                 </p>
<p><strong>Status:</strong>                 {{ $stripeData['subscription']?->status ?? 'No Active
Subscription' }}                 </p>             </div>             <hr class="my-4">             <h3 class="font-semibold
mb-2">Recent Invoices</h3>             <table class="min-w-full text-sm border">                 <thead class="bg-
gray-100">                     <tr>                         <th class="p-2 text-left">Date</th>                         <th class="p-2
```

```

text-left">Amount</th>                <th class="p-2 text-left">Status</th>                </tr>
</thead>                <tbody>                @foreach($stripeData['invoices'] as $invoice)                <tr>
<td class="p-2">{{ date('Y-m-d', $invoice->created) }}</td>                <td class="p-2">{{
number_format($invoice->amount_paid / 100, 2) }}</td>                <td class="p-2">{{
>status) }}</td>                </tr>                @endforeach                </tbody>                </table>
@endif                </div>                </div> </x-filament::page>

```

□ 7. Link “Stripe History” Page to User Table

Edit your `UserResource.php`:

```

->actions([                Tables\Actions\ViewAction::make('stripe')                ->label('View Stripe')                -
>url(fn($record) => route('filament.pages.stripe-history', $record))                ->openUrlInNewTab(), ])

```

□ Now each user in `/admin/users` will have a “**View Stripe**” button linking to their live billing details.

□ 8. Permissions

Only `is_admin` can access this page.

Optional IP whitelist:

```

if (!in_array(request()->ip(), ['YOUR_STATIC_IP'])) abort(403);

```

□ 9. Bonus: Add Total Spend Field

In your Filament table:

```

TextColumn::make('lifetime_value')                ->label('Lifetime $')                ->getStateUsing(fn($record) => '$' .
number_format($record->invoices()->sum('amount_paid') / 100, 2                ));

```

(Or fetch directly from Stripe if you want exact amounts.)

□ 10. FINAL RESULT

When you log in to **Pixrup Admin** → **Users** → **View Stripe**:
You'll see a live snapshot:

- **Customer ID (click to open in Stripe)**
- **Current plan + status**
- **Next renewal date**
- **Recent invoices**
- **Total paid to date**

It updates automatically via Stripe's API — no manual syncing.

SaaS Growth Timeline Widget for Filament Admin

□ □ Step 1 – Create the Widget

Run this command:

```
php artisan make:filament-widget SubscriptionTimeline
```

□ File: `app/Filament/Widgets/SubscriptionTimeline.php`

```
namespace App\Filament\Widgets; use App\Models\User; use Filament\Widgets\ChartWidget; use
Illuminate\Support\Facades\DB; class SubscriptionTimeline extends ChartWidget {     protected static
?string $heading = 'Subscriber Growth by Plan (Last 6 Months)';     protected int | string | array
$columnSpan = 'full';     protected function getData(): array     {         // Generate data for last 6
months         $months = collect(range(0, 5))->map(fn($i) => now()->subMonths($i)->format('Y-m'));
// Fetch counts by plan and month         $data = [             'starter' => [],             'pro' => [],
'enterprise' => [],         ];         foreach ($months as $month) {             $data['starter'][] =
User::where('plan_tier', 'PRICE_STARTER')                 ->whereMonth('created_at', substr($month, 5, 2))
->whereYear('created_at', substr($month, 0, 4))                 ->count();             $data['pro'][] =
User::where('plan_tier', 'PRICE_PRO')                 ->whereMonth('created_at', substr($month, 5, 2))
->whereYear('created_at', substr($month, 0, 4))                 ->count();             $data['enterprise'][] = User::where('plan_tier', 'PRICE_ENTERPRISE')
->whereMonth('created_at', substr($month, 5, 2))                 ->whereYear('created_at', substr($month, 0,
4))                 ->count();         }         return [             'datasets' => [                 [
'label' => 'Starter',                 'data' => array_reverse($data['starter']),                 'borderColor' => '#3b82f6',                 'fill' => false,                 ],                 [
'label' => 'Pro',                 'data' => array_reverse($data['pro']),                 'borderColor' => '#f59e0b',                 'fill' => false,                 ],                 [
'label' => 'Enterprise',                 'data' => array_reverse($data['enterprise']),                 'borderColor' => '#10b981',                 'fill' => false,                 ],             ],         ]     }
```

```
'labels' => $months->reverse()->values()->toArray(),  
string { return 'line'; } }  
]; } protected function getType():
```

☐ Step 2 – Add Styling and Colors

The line chart will show:

- ☐ **Blue line** → Starter users
- ☐ **Amber line** → Pro users
- ☐ **Green line** → Enterprise users

Each point shows total new signups for that month.
Chart.js (built into Filament) automatically animates the graph.

☐ ☐ Step 3 – Register the Widget

Edit your **Filament Dashboard Page**:

☐ `app/Filament/Pages/Dashboard.php`

Add:

```
protected function getWidgets(): array { return [  
UsageChart::class, StripeRevenue::class, ActiveStats::class,  
SubscriptionTimeline::class, // ☐ Add this line  
]; }
```

☐ Step 4 – Optional Data Optimization

If you want smoother charts later, Sagit can cache this widget:

```
protected static bool $isLazy = true; protected static ?string $pollingInterval = '60s';
```

This refreshes data every minute — perfect for live dashboards.

□ Step 5 – What You’ll See

When you open `/admin`, you’ll now have:

- Top row: Stats cards (Active Users, Enterprise Users, Monthly Revenue)
- Second row: UsageChart (bar graph)
- Third row: □ **SubscriptionTimeline** (growth lines)

You’ll visually see your platform’s user base expanding month by month, separated by pricing tier.

□ Result

Your admin area now includes:

Feature	Function
ActiveStats	Real-time MRR, users per plan
UsageChart	Documents usage analytics
StripeRevenue	Live MRR from Stripe
StripeHistory	Per-user payment history
SubscriptionTimeline	SaaS growth graph (6-month trend)

You’ve officially turned Pixrup into a **SaaS-grade analytics HQ**, on par with tools like Paddle, ProfitWell, or Stripe Sigma — but built entirely in your Laravel app for \$0.

final layer that completes your full **Pixrup SaaS Admin Intelligence System**:

The **Team Performance & Feature Usage Widget** ☐

This tracks which AI tools users love most (PDF exports, AI renders, voice readouts, etc.), how often they're used, and which plans generate the most engagement.

It transforms your admin into a *product-intelligence dashboard* like Notion Analytics or Mixpanel — all self-hosted and free.

☐ ☐ **1** ☐ ☐ **PURPOSE**

To show, in real time:

- ☐ How many times each AI feature (e.g., “AI Render”, “PDF Export”, “Voice Summary”) was used this month.
 - ☐ Which plans (Starter, Pro, Enterprise) use each feature most.
 - ☐ Who your power users are (great for upselling or case studies).
-

☐ **2** ☐ ☐ **DATABASE PREP**

Add a simple tracking table (if not already logging usage):

```
Schema::create('feature_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('feature'); // e.g. 'pdf_export', 'ai_render', 'tts_summary'
    $table->timestamps();
});
```

Every time a user triggers a feature, you log it:

```
FeatureLog::create([
    'user_id' => auth()->id(),
    'feature' => 'ai_render',
]);
```

3 CREATE THE WIDGET

php artisan make:filament-widget FeatureUsageWidget

app/Filament/Widgets/FeatureUsageWidget.php

```
namespace App\Filament\Widgets; use App\Models\FeatureLog; use App\Models\User; use
Filament\Widgets\ChartWidget; use Illuminate\Support\Facades\DB; class FeatureUsageWidget extends
ChartWidget { protected static ?string $heading = 'AI Feature Usage (Last 30 Days)'; protected int
| string | array $columnSpan = 'full'; protected function getData(): array { $features =
['pdf_export', 'ai_render', 'tts_summary', 'branding_update']; $counts = []; foreach
($features as $f) { $counts[$f] = FeatureLog::where('feature', $f) -
>where('created_at', '>=', now()->subDays(30)) ->count(); } return [
'datasets' => [ [ 'label' => 'Feature Usage',
'data' => array_values($counts), 'backgroundColor' =>
['#3b82f6', '#f59e0b', '#10b981', '#ef4444'], ], 'labels' =>
array_map('ucwords', str_replace('_', ' ', array_keys($counts))), ]; } protected function
getType(): string { return 'bar'; } }
```

4 ADD POWER-USER TABLE (OPTIONAL)

If you want to list your **top 10 users** by feature usage, Sagit can add this block below the chart:

```
$topUsers = User::select('name') ->selectRaw('COUNT(feature_logs.id) as total') -
>join('feature_logs', 'users.id', '=', 'feature_logs.user_id') ->where('feature_logs.created_at', '>=',
now()->subDays(30)) ->groupBy('users.id') ->orderByDesc('total') ->limit(10) ->get();
```

Then display in Blade:

```
<h3 class="font-semibold mt-6">Top 10 Power Users</h3> <table class="min-w-full text-sm border">
<thead><tr><th>Name</th><th>Actions</th></tr></thead> <tbody> @foreach($topUsers as $user)
<tr><td>{{ $user->name }}</td><td>{{ $user->total }}</td></tr> @endforeach </tbody> </table>
```

5 REGISTER THE WIDGET

Edit your main dashboard file:

app/Filament/Pages/Dashboard.php

```
protected function getWidgets(): array {    return [        ActiveStats::class,        UsageChart::class,        StripeRevenue::class,        SubscriptionTimeline::class,        FeatureUsageWidget::class, // New analytics layer    ]; }
```

6 OPTIONAL — FILTER BY PLAN

To see which plan uses which feature most, add:

```
$planUsage = DB::table('feature_logs')->join('users', 'users.id', '=', 'feature_logs.user_id')->select('users.plan_tier', DB::raw('count(*) as total'))->groupBy('users.plan_tier')->pluck('total', 'users.plan_tier');
```

You can show this as a **pie chart** inside the widget:

```
return [    'datasets' => [        'label' => 'Usage by Plan',        'data' => $planUsage->values(),        'backgroundColor' => ['#3b82f6', '#f59e0b', '#10b981', '#6366f1'],        'labels' => $planUsage->keys(),    ],];
```

7 DATA FLOW OVERVIEW

Event

Logged In Table

Reflected In Widget

Event	Logged In Table	Reflected In Widget
User exports PDF	feature_logs (pdf_export)	Bar chart updates
User uses AI render	feature_logs (ai_render)	Bar chart updates
User listens to TTS summary	feature_logs (tts_summary)	Bar chart updates
User customizes branding	feature_logs (branding_update)	Bar chart updates

8 AUTOMATED METRICS

You can extend this with AI alerts later:

- “Top Feature this Month: AI Render (82% of users used it)”
- “Enterprise users used Branding 4× more than Pro users”
- “Feature TTS Summary usage up +12% vs. last month”

These metrics can appear as Filament `StatsOverview` cards easily.

9 OPTIONAL BONUS

Add this artisan command for monthly reports:

```
php artisan make:command GenerateFeatureReport
class GenerateFeatureReport extends Command {
    protected $signature = 'report:features';
    protected $description = 'Generate monthly feature usage report';
    public function handle() {
        $features = DB::table('feature_logs')->select('feature', DB::raw('COUNT(*) as total'))->whereBetween('created_at', [now()->startOfMonth(), now()])->groupBy('feature')->pluck('total', 'feature');
        $this->info("Monthly Usage Report:\n" . $features);
    }
}
```

You can later automate it to email you on the 1st of every month.

☐ FINAL RESULT

Your **Pixrup Admin** now has:

Layer	Description
☐ ActiveStats	Live MRR + user counts
☐ SubscriptionTimeline	Growth of plans (6 months)
☐ StripeRevenue	Real-time MRR from Stripe
☐ StripeHistory	Per-user billing details
☐ ☐ FeatureUsageWidget	Shows feature adoption & engagement
☐ Power Users List	Top-performing users
☐ Cron Reports	Monthly usage summaries

this is the **crowning jewel** of your analytics stack:
the **Feature Heatmap Widget** — your visual map of how every Pixrup feature performs **per plan tier** (Starter, Pro, Enterprise).

This is the same type of product intelligence dashboard big SaaS companies use internally — think Notion, ClickUp, or Figma — showing exactly which tools are driving adoption and retention.

□ 1 □ □ GOAL

Create a dynamic **heatmap chart** in your admin dashboard that answers:

“Which features do my users love most, broken down by plan?”

For example:

Feature →	Starter	Pro	Enterprise
PDF Exports	□□	□□□	□□
AI Renders	□	□□	□□□□
Voice Summary	□	□□	□□□
Branding Updates	□	□	□□□

□ □ 2 □ □ DATABASE SETUP

We'll use your existing `feature_logs` table from the last widget.

Make sure it includes:

```
Schema::create('feature_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('feature'); // e.g. 'pdf_export', 'ai_render', 'tts_summary', 'branding_update'
    $table->timestamps();
});
```

3 CREATE THE WIDGET

php artisan make:filament-widget FeatureHeatmapWidget

File: app/Filament/Widgets/FeatureHeatmapWidget.php

```
namespace App\Filament\Widgets; use App\Models\FeatureLog; use App\Models\User; use
Filament\Widgets\Widget; use Illuminate\Support\Facades\DB; class FeatureHeatmapWidget extends Widget {
    protected static ?string $heading = 'Feature Engagement Heatmap (Per Plan Tier)';
    protected static string $view = 'filament.widgets.feature-heatmap-widget';
    protected int | string | array $columnSpan = 'full';
    public function getHeatmapData() {
        $features = ['pdf_export', 'ai_render', 'tts_summary', 'branding_update'];
        $plans = ['PRICE_STARTER', 'PRICE_PRO', 'PRICE_ENTERPRISE'];
        $data = [];
        foreach ($features as $feature) {
            foreach ($plans as $plan) {
                $data[$feature][$plan] = FeatureLog::join('users', 'feature_logs.user_id', '=', 'users.id')
                    ->where('feature_logs.feature', $feature)
                    ->where('users.plan_tier', $plan)
                    ->where('feature_logs.created_at', '>=', now()->subDays(30))
                    ->count();
            }
        }
        return $data;
    }
```

4 CREATE THE BLADE VIEW

resources/views/filament/widgets/feature-heatmap-widget.blade.php

```
<x-filament::widget> <x-filament::card> <h2 class="text-lg font-bold mb-4">Feature Engagement by
Plan</h2> @php $data = (new \App\Filament\Widgets\FeatureHeatmapWidget)->getHeatmapData();
$plans = ['PRICE_STARTER' => 'Starter', 'PRICE_PRO' => 'Pro', 'PRICE_ENTERPRISE' => 'Enterprise'];
@endphp <table class="min-w-full text-sm text-center border-collapse"> <thead class="bg-gray-
100"> <tr> <th class="border p-2">Feature</th> @foreach($plans as $key =>
$name) <th class="border p-2">{{ $name }}</th> @endforeach </tr>
```

```

</thead>      <tbody>          @foreach($data as $feature => $planCounts)      <tr>          <td
class="border p-2 font-semibold capitalize">{{ str_replace('_', ' ', $feature) }}</td>
@foreach($plans as $key => $name)          @php          $count = $planCounts[$key];
$color = match(true) {          $count == 0 => 'bg-gray-200',          $count < 5 => 'bg-
green-100',          $count < 15 => 'bg-green-300',          $count < 30 => 'bg-green-500
text-white',          default => 'bg-green-700 text-white',          };
@endphp          <td class="border p-2 {{ $color }}">{{ $count }}</td>          @endforeach
</tr>          @endforeach      </tbody>      </table>      <p class="text-xs text-gray-500 mt-4">Updated
every 30 days. Darker = higher engagement.</p>      </x-filament::card> </x-filament::widget>

```

□ 5 □ □ HOW IT WORKS

- Pulls 30 days of feature usage logs.
- Groups by plan tier and feature name.
- Builds a color intensity grid (light = low use, dark = heavy use).
- Updates automatically whenever the admin dashboard loads.

Each cell's color strength represents activity level.

□ 6 □ □ ADD TO DASHBOARD

In app/Filament/Pages/Dashboard.php:

```

protected function getWidgets(): array {      return [          ActiveStats::class,
UsageChart::class,          StripeRevenue::class,          SubscriptionTimeline::class,
FeatureUsageWidget::class,          FeatureHeatmapWidget::class, // □ Final analytics layer      ]; }

```

□ 7 □ □ OPTIONAL ENHANCEMENTS

Add-on	Purpose
Plan filter dropdown	Quickly filter heatmap by one tier (e.g., show Enterprise only)
Date range picker	Change 30 days → 90 days
Export CSV button	Export data for investor reporting
Hover tooltips	Show “23 total uses – 12 unique users” when hovering over cells

Sagit can implement all of these easily using Filament’s Livewire-powered interactivity.

□ 8 □ □ HOW YOU'LL USE IT

You’ll open `/admin` and instantly see:

- Which Pixrup features are driving engagement (e.g. “AI Render used 200× this month”).
 - Which plans use which tools most (“Enterprise users love Branding, Pro users use Renders”).
 - What’s underused (great for training, marketing, or upgrades).
-

□ FINAL ADMIN SUITE SUMMARY

Module	Function
□ ActiveStats	High-level KPIs (users, MRR)
□ StripeRevenue	Live Stripe income
□ SubscriptionTimeline	Growth over time

Module	Function
☐ UsageChart	Document creation trends
☐ ☐ FeatureUsageWidget	Most-used Pixrup features
☐ FeatureHeatmapWidget	Visual plan-tier engagement map
☐ StripeHistory Page	Individual payment & subscription details

☐ **End Result**

Your `/admin` dashboard becomes a true **AI SaaS Control Center**:

- ☐ Live MRR tracking
- ☐ Tier-based analytics
- ☐ Feature engagement heatmap
- ☐ Growth timeline
- ☐ Power-user leaderboard
- ☐ Stripe-connected billing insights

All **built on your own Laravel system** — no external dashboards, no cost, fully private.